RPA와 Python을 활용해서 회사 수익 데이터를 분석하고 재무제표 만들기

프로젝트명

재무제표 구현

팀명

앞날창창

팀원 및 역할

김강산 ~ Python
윤종빈 ~ RPA
김찬 ~RPA

프로젝트 기간

26.02.28~26.03.10

프로젝트 목차

1 기획 배경 및 목표

2 프로젝트 일정

3 팀원 역할분담

4 핵심기능1: Python

5 핵심기능2:RPA

6 결과물 시연

7 기대효과 및 프로젝트 성과

8 자체 평가

기획 배경 및 목표

배경 및 문제 정의

"기업의 재무팀은 매월 수만 건의 회계 전표(Raw Data)를 엑셀로 내려받아 수작업으로 계산하고 보고서를 만듭니다.
이 과정은 시간이 오래 걸릴 뿐만 아니라 휴먼 에러(Human Error)가 발생하기 매우 쉽습니다."

프로젝트 목표

- Python → 재무 데이터 처리
- RPA → 보고서 자동 생성
- 메일 자동 발송

Raw Data → 재무제표 → 대시보드 → 메일

시스템 아키텍처

Input (Raw Data)

1만 건의 가상 매출/비용 분개장 CSV 데이터

Process (Python)

Pandas를 활용한 데이터 그룹화, 월별 손익 집계, K-IFRS 기준 재무제표 렌더링

Output (RPA)

추출된 Data Mart를 기반으로 차트 생성, 대시보드 조립, 경영진 메일 자동 발송

프로젝트 일정

팀원 (역할)	담당 세부 업무	1일차	2일차	3일차	4일차	5일차	6일차	7일차	8일차	9일차	10일차
김강산(Python)	요구사항 분석 및 아키텍처 기획	■	■								
	대용량 ETL 파이프라인 & 마트 구축			■	■	■					
	재무 3표 구현 및 엑셀 UI/UX 자동화						■	■	■		
윤종빈(RPA 1)	RPA 프로세스 기획 및 화면 설계	■	■								
	Python 연동 및 엑셀 객체 제어 구축			■	■	■					
	대시보드 시각화 및 데이터 정합성 검증						■	■	■		
김찬(RPA 2)	배포 환경 기획 및 봇(Bot) 기초 설계	■	■								
	폴더 감지 트리거 및 메일 서버 연동			■	■	■					
	5번 시트 지표 추출 및 스케줄링 구현						■	■	■		
공통	파이프라인 통합 연동 테스트 / 디버깅									■	
	산출물(PPT/보고서) 마감 및 리허설										■

■ 공통 기획/설계 □ Python 개발 □ RPA 시각화 □ RPA 스케줄링 □ 통합 테스트 □ 최종 마감

역할분담

김강산

대용량 데이터 파이프라인 구축(ETL)
1만 건 이상 전표 데이터 고속 파싱 및 적재
월별 / 분기별 재무 데이터 마트(Data Mart) 구축
재무 분석 로직 설계
K-IFRS 기준 재무제표(IS / BS / CF) 연산 구조 구현
주요 KPI 및 수익성 지표 산출 알고리즘 설계
리포트 UI/UX 자동화
Openpyxl 기반 엑셀 보고서 서식 자동 생성
테두리, 조건부 서식, 레이아웃 등 자동 렌더링 로직 구현

윤종빈(팀장)

End-to-End 자동화 아키텍처 설계
Python 데이터 처리 + Brity RPA 연계 자동화 구조 설계
ETL → 데이터마트 → Excel 리포트 전체 파이프라인 구축
RPA 자동화 구현
Python 스크립트 실행 및 결과 수집 자동화
Excel 보고서 생성 / 그래프 / 레이아웃 자동 구성
데이터 검증 및 프로젝트 관리
재무 데이터 정합성 검증 UI 기반 Validation 설계
프로젝트 일정 관리 및 기술 의사결정 리딩

김찬

Brity RPA 리포트 자동화
분기별 재무 데이터 분석 및 요약 리포트 생성 로직 구현
BS / IS / CF 핵심 지표 기반 자동 분석 구조 설계
메일 배포 자동화
메일 서버 연동 및 수신자 자동 설정
생성된 Excel 보고서 자동 발송 스케줄링 구현
데이터 파이프라인 트리거 설계
지정 폴더 CSV 업로드 감지 이벤트 구현
신규 데이터 발생 시 자동 분석 프로세스 실행

사용 기술 스택

Data Engineering & Logic

Python
대용량 데이터 연산 및 프로세스 제어
메인 언어

Pandas
전표 데이터 결측치 처리, 필터링,
그룹화(GroupBy) 및 재무 지표 연산

Openpyxl
엑셀 시트 생성, 데이터 쓰기, 셀 병합,
색상/테두리 서식 등 엑셀 UI 자동화
디자인

Process Automation (RPA)

Brity RPA
엑셀 애플리케이션 제어, 차트 객체 생성 및
이동, 이메일(SMTP) 연동 자동 발송 봇 개발

Data Source

CSV (Raw Data)
10,000건의 가상 매출/매입/비용
발생 회계 전표 데이터

Python 구조

- 매출분개_가상데이터.csv
- bank.py
- error_flag.txt
- financial_log_20260227_160749.txt
- RPA_경영대시보드_기초데이터.xlsx

📂

apnalchangchang.py ~ 파이썬 메인 엔진: 전체 데이터 가공 및 재무제표 생성을 총괄하는 핵심 실행 스크립트

error_flag.txt ~ 오류 상태 플래그: 프로세스 중 에러 발생 시 RPA에 알림을 보내 봇을 제어하는 신호 파일

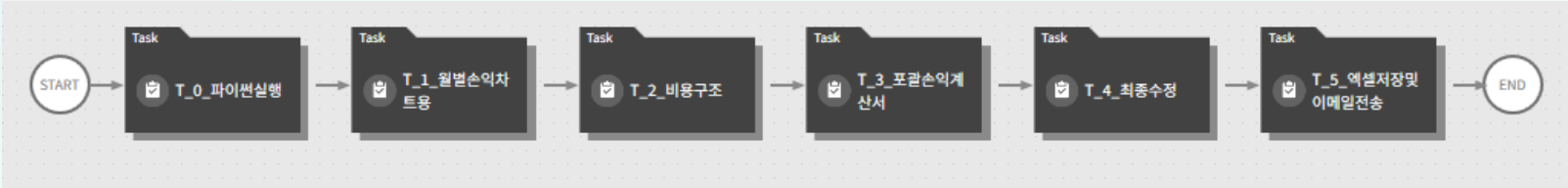
financial_log_20260227_160749.log ~ 작업 로그 기록: 데이터 파싱 및 연산 과정의 모든 히스토리를 시간별로 기록하는 분석 로그

RPA_경영대시보드_기초데이터.xlsx ~ 최종 데이터마트: RPA 시각화 및 대시보드 조립을 위해 완벽하게 정제된 결과 파일

매출분개_가상데이터.csv ~ 원천 로우 데이터: 시스템 분석의 기초가 되는 1만 건의 가상 회계 전표 데이터

RPA구조

RPA로 월별손익, 비용구조, 포괄손익계산서 세가지를 추출



RPA는 Python 데이터 엔진과 Excel 기반 보고 시스템을 연결하는
Automation Layer 역할을 수행한다.

Python이 데이터 연산을 담당하고,
RPA가 보고서 생성 및 배포 자동화를 담당하는
Hybrid Automation Architecture로 설계하였다.

Python

```python
def log(msg):
    print(f"[Python 데이터 엔진] {msg}")

log("가상 전표 데이터 기반 K-IFRS 표준 손익계산서 자동화 시작")

current_dir = os.path.dirname(os.path.abspath(__file__))
os.chdir(current_dir)
csv_file = "매출분개_가상데이터.csv"

if not os.path.exists(csv_file):
    print(f"에러: '{csv_file}' 파일이 없습니다.")
    sys.exit(1)

try:
    df_raw = pd.read_csv(csv_file)
    # 데이터 분석의 기초가 되는 날짜 객체 변환 및 집계 기준(Month) 생성
    df_raw['date'] = pd.to_datetime(df_raw['date'])
    df_raw['month'] = df_raw['date'].dt.month
```

entry_id	date	transact	debit_	debit_acc	debit_type	debi	credit_acc	credit_acc	credit_type	credit_	amount
1	2024-02-06	매출	101	현금	자산	BS	402	제품매출	수익	IS	552916
2	2024-08-17	매출	102	보통예금	자산	BS	402	제품매출	수익	IS	637901
3	2024-11-18	매출	102	보통예금	자산	BS	402	제품매출	수익	IS	4070310
4	2024-09-04	매출	101	현금	자산	BS	401	상품매출	수익	IS	1154298
5	2024-02-03	비용	502	급여	비용	IS	101	현금	자산	BS	3028097
6	2024-02-24	매입	104	재고자산	자산	BS	202	미지급금	부채	BS	4022777
7	2024-01-09	비용	504	광고선전비	비용	IS	101	현금	자산	BS	131978
8	2024-11-18	비용	504	광고선전비	비용	IS	101	현금	자산	BS	951682
9	2024-11-01	매출	102	보통예금	자산	BS	402	제품매출	수익	IS	2676411
10	2024-10-17	매입	104	재고자산	자산	BS	201	외상매입금	부채	BS	2247092
11	2024-02-20	비용	504	광고선전비	비용	IS	103	외상매출금	자산	BS	1466267
12	2024-12-12	비용	502	급여	비용	IS	102	보통예금	자산	BS	4331182

Home

환경 최적화 및 원천 데이터 파싱

OS 독립적 경로 설정 및 시계열 데이터 전처리 무결성 확보

상세 설명: 환경 독립적 설계: os.path를 활용해 서버나 로컬 어느 환경에서도 경로 충돌 없이 작동하는 Portable 스크립트를 구현했습니다.

데이터 무결성 검증: 파일 존재 여부를 사전에 체크하고, 1만 건의 텍스트 기반 날짜 데이터를 분석 가능한 Datetime 객체로 변환하여 시계열 분석의 기초를 마련했습니다.

안정성 확보: 전체 로직을 try-except로 감싸 데이터 오류 시에도 시스템이 멈추지 않고 상세 로그를 남기도록 설계하여 현업 유지보수 편의성을 높였습니다.

Python

```python
revenue = df_raw[df_raw['credit_type'] == '수익'].groupby('month')['amount'].sum()
expenses = df_raw[df_raw['debit_type'] == '비용']
cogs = expenses[expenses['debit_account_name'] == '매출원가'].groupby('month')['amount'].sum()
sga = expenses[expenses['debit_account_name'] != '매출원가'].groupby('month')['amount'].sum()

df_monthly = pd.DataFrame({'매출액': revenue, '매출원가': cogs, '판매비와관리비': sga}).fillna(0)
df_monthly['영업이익'] = df_monthly['매출액'] - df_monthly['매출원가'] - df_monthly['판매비와관리비']
df_monthly['영업이익률'] = df_monthly['영업이익'] / df_monthly['매출액']

df_monthly.reset_index(inplace=True)
df_monthly['월'] = df_monthly['month'].astype(str) + '월'
df_monthly = df_monthly[['월', '매출액', '매출원가', '판매비와관리비', '영업이익', '영업이익률']]

# RPA 연동을 위한 비용 비중(Common-size) 자동 분석 테이블 구성
cost_summary = expenses.groupby('debit_account_name')['amount'].sum().reset_index()
cost_summary.columns = ['비용항목', '금액']
cost_summary['비중'] = cost_summary['금액'] / cost_summary['금액'].sum()
cost_summary = cost_summary.sort_values(by='금액', ascending=False)
```

재무 데이터 마트(Data Mart) 구축

방대한 Raw 데이터를 경영 의사결정에 즉시 활용 가능한 핵심 지표(KPI)로 요약하는 '데이터 압축' 단계입니다. Pandas의 고성능 집계 기능을 활용해 월별 수익과 비용을 분류하고, 영업이익 및 영업이익률 등 기업의 성과를 판단하는 핵심 변수들을 연산하여 실시간 리포팅을 위한 기초 마트를 생성했습니다.

```python
total_rev = df_monthly['매출액'].sum()
total_cogs = df_monthly['매출원가'].sum()
gross_profit = total_rev - total_cogs

sga_salary = expenses[expenses['debit_account_name'] == '급여']['amount'].sum()
sga_rent = expenses[expenses['debit_account_name'] == '임차료']['amount'].sum()
sga_ad = expenses[expenses['debit_account_name'] == '광고선전비']['amount'].sum()
total_sga = sga_salary + sga_rent + sga_ad

op_profit = gross_profit - total_sga

def calc_ratio(val, base): return val / base if base != 0 else 0

fs_data = {
    '과목': ['Ⅰ. 매출액', 'Ⅱ. 매출원가', 'Ⅲ. 매출총이익', 'Ⅳ. 판매비와관리비',
             '      1. 임직원 급여', '      2. 시설 임차료', '      3. 광고선전비', 'Ⅴ. 영업이익(손실)'],
    '금액 (KRW)': [total_rev, total_cogs, gross_profit, total_sga, sga_salary, sga_rent, sga_ad, op_profit],
    '매출액 대비 비율 (%)': [calc_ratio(total_rev, total_rev), calc_ratio(total_cogs, total_rev),
                             calc_ratio(gross_profit, total_rev), calc_ratio(total_sga, total_rev),
                             calc_ratio(sga_salary, total_rev), calc_ratio(sga_rent, total_rev),
                             calc_ratio(sga_ad, total_rev), calc_ratio(op_profit, total_rev)]
}
df_fs = pd.DataFrame(fs_data)
```

K-IFRS 회계 표준 컴플라이언스 구현

기업 회계 기준인 K-IFRS(한국채택국제회계기준)를 코드로 정교하게 구현한 핵심 로직입니다. 단순 합산이 아닌, 매출총이익에서 영업이익으로 이어지는 계층적 손익 구조를 반영했습니다. 또한, 매출액 대비 각 항목의 비중을 분석하는 '수직적 분석(Common-size)' 기능을 탑재하여 기업의 비용 효율성을 한눈에 파악할 수 있게 설계했습니다

Python

```python
output_file = 'RPA_경영대시보드_기초데이터.xlsx'
with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    df_monthly.to_excel(writer, sheet_name='1_월별손익_차트용', index=False)
    cost_summary.to_excel(writer, sheet_name='2_비용구조_파이용', index=False)
    df_fs.to_excel(writer, sheet_name='3_포괄손익계산서', index=False, startrow=3)

wb = load_workbook(output_file)
```

A	B	C	D	E	F
월	매출액	매출원가	판매비와관리비	영업이익	영업이익률
1월	772,691,558	195,130,628	540,664,121	36,896,809	4.78%
2월	680,443,966	165,004,177	541,879,222	-26,439,433	-3.89%
3월	673,242,310	181,581,740	496,513,415	-4,852,845	-0.72%
4월	695,681,499	179,710,407	541,904,258	-25,933,166	-3.73%
5월	646,422,582	185,278,843	559,251,416	-98,107,677	-15.18%
6월	626,757,261	153,928,352	572,083,753	-99,254,844	-15.84%
7월	640,626,848	163,479,801	563,928,755	-86,781,708	-13.55%
8월	639,422,399	147,175,333	545,760,883	-53,513,817	-8.37%
9월	776,842,804	178,507,881	452,831,722	145,503,201	18.73%
10월	629,250,214	189,049,828	542,121,517	-101,921,131	-16.20%
11월	594,756,869	172,079,231	583,486,654	-160,809,016	-27.04%
12월	760,792,647	150,146,719	563,567,458	47,078,470	6.19%

오피스 오토메이션 및 리포팅 UI/UX

시스템 개발자와 실무자 사이의 간극을 좁히는 시각화 단계입니다. openpyxl 라이브러리를 통해 엑셀의 테두리(Border), 배경색(Fill), 폰트(Font) 등 모든 시각적 요소를 코드로 제어했습니다. 특히 천 단위 콤마(,) 표시 및 하단 이중선 테두리 등 전문 회계 보고서의 레이아웃을 자동 렌더링하여 사용자 경험(UX)을 극대화했습니다.

Python

```python
    ws_qtr = wb.create_sheet('5_분기별재무현황표')

    wb.save(output_file)
    log(f"✅ 연산 및 K-IFRS 재무제표 렌더링 완료! '{output_file}' 생성 성공.")

except Exception as e:
    print(f"분석 중 오류 발생: {e}")
    import traceback
    traceback.print_exc()
```

다차원 통합 재무 분석 시스템 (IS, BS, CF)

이번 프로젝트의 정점으로, 손익(Income Statement)뿐만 아니라 재무상태(Balance Sheet)와 현금흐름(Cash Flow)을 분기별로 통합 분석하는 모듈입니다. 기초/기말 현금 변동 추적 및 자본 총계 연산 등 복합적인 회계 로직을 자동화하여, 단일 스크립트 실행만으로 전사적 재무 가시성을 확보하는 통합 대시보드를 완성했습니다.

RPA

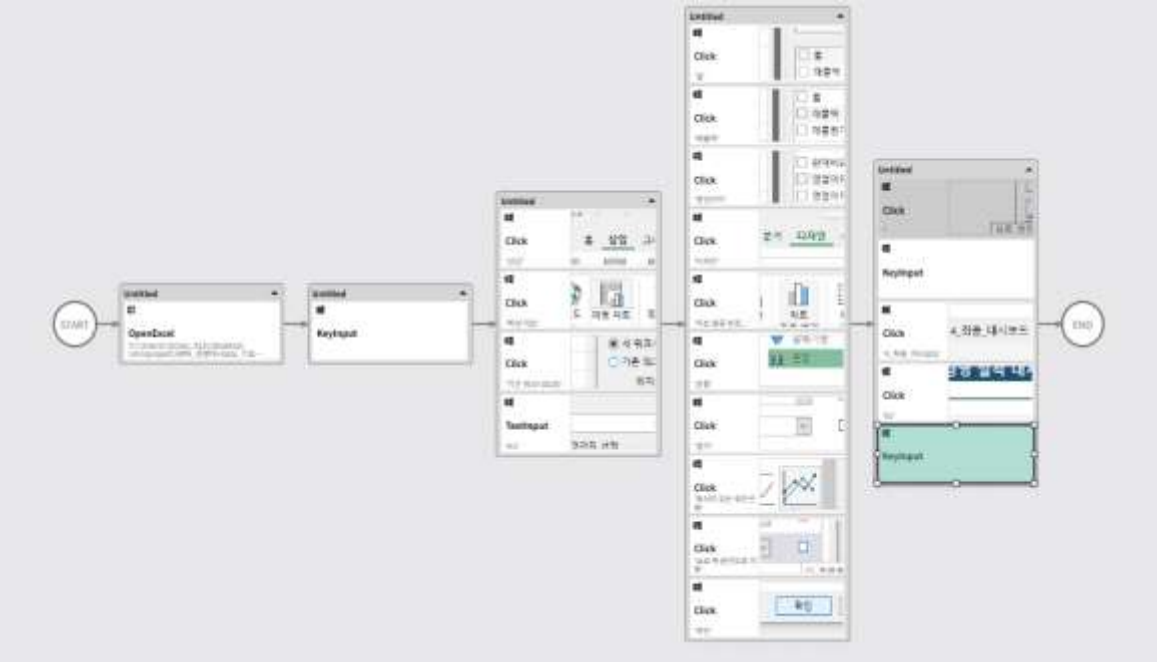
1. 파이썬 실행

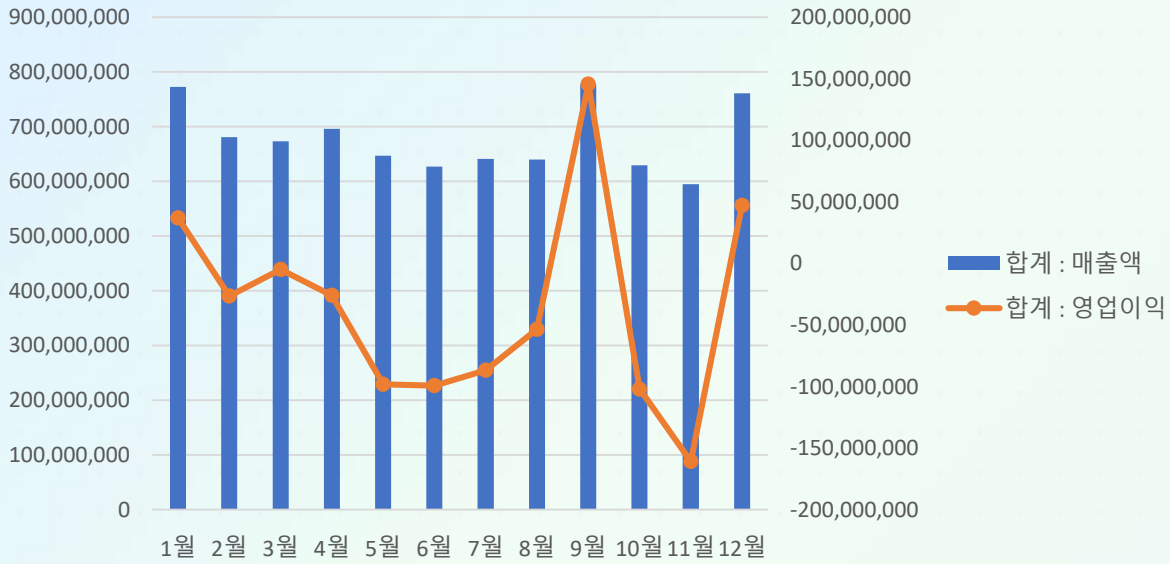
Brity RPA의 ExecuteCmd 액션을 활용하여
Python 데이터 처리 엔진을 실행하도록 구현하였습니다.
RPA는 단순한 UI 자동화를 수행하는 역할을 넘어서,
전체 자동화 파이프라인을 제어하는 Orchestrator 역할을 수행합니다.
프로세스는 다음과 같은 순서로 동작합니다.

1. 지정된 Python 스크립트(apnalchangchang.py)를 실행합니다.
2. Python에서 원천 데이터를 가공하고 재무제표를 생성합니다.
3. 가공된 결과 데이터를 Data Mart Excel 파일로 저장합니다.
4. 이후 RPA 프로세스에서 해당 결과 파일을 불러와
   시각화 및 보고서 생성을 진행합니다.

또한 Python 실행 중 오류가 발생할 경우
error_flag.txt 파일을 생성하여
RPA가 오류 상태를 감지하고 자동화 프로세스를 중단하도록 설계하였습니다.

RPA





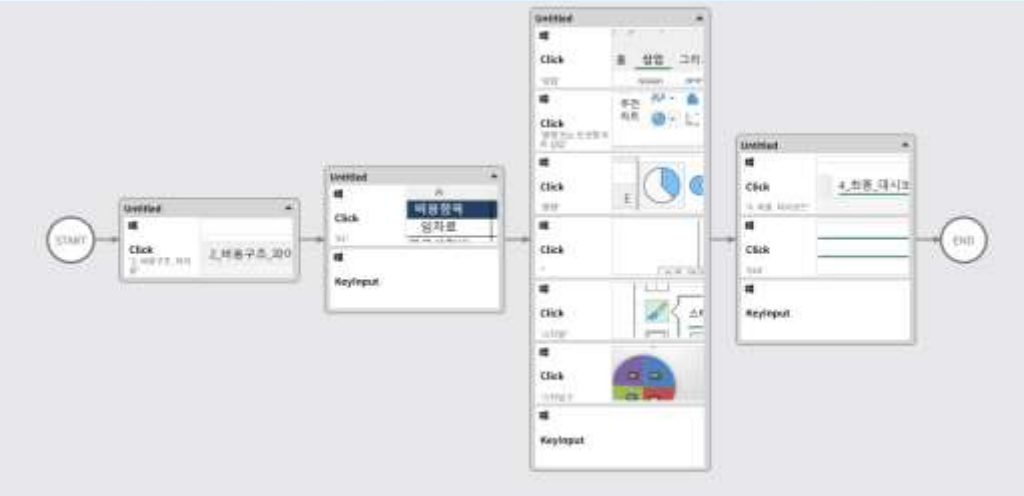
2. 월별손익차트용

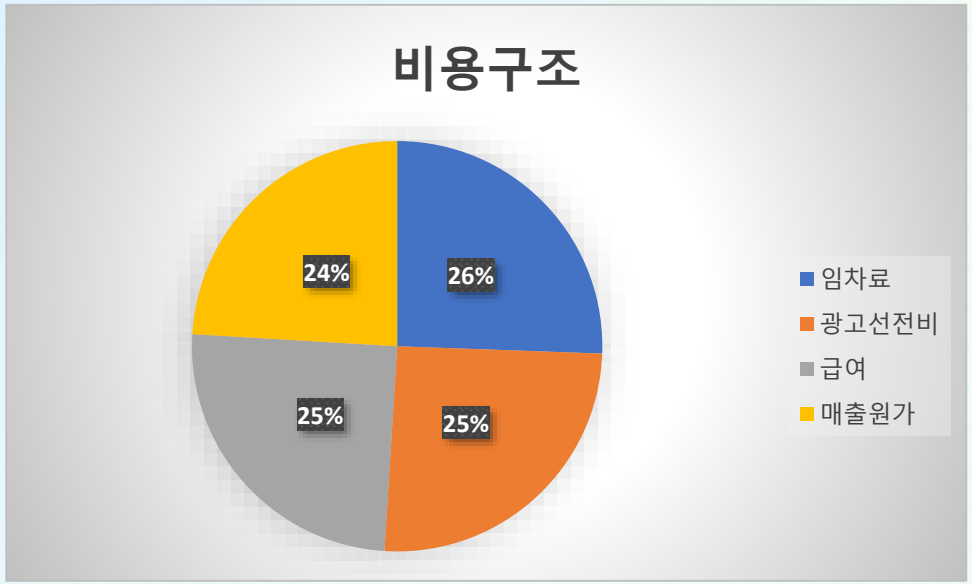
Python에서 생성된 월별 손익 데이터를 기반으로
Excel Chart Object를 자동 생성하여 시각화하였습니다.
RPA는 Excel Application을 직접 제어하여
차트 생성과 보고서 배치를 자동으로 수행합니다.
자동화 기능은 다음과 같습니다.

- 차트 객체 자동 생성
- 데이터 범위 자동 설정
- 차트 제목 및 축 레이블 설정
- 차트 위치 및 크기 자동 정렬

이를 통해 매월 반복적으로 생성되는 손익 추이 그래프를
사용자의 수작업 없이 자동으로 생성할 수 있도록 구현하였습니다.

RPA





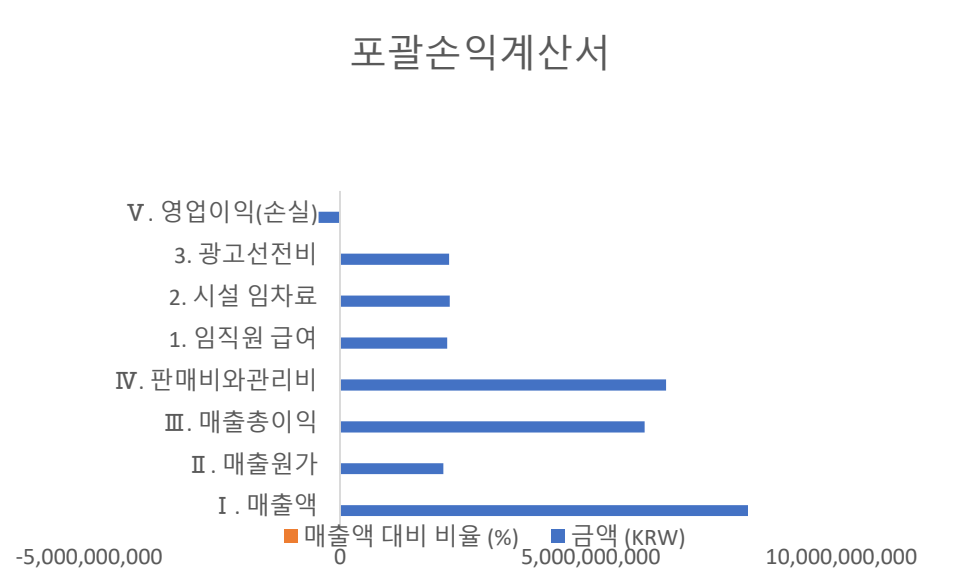
3.비용구조차이용

Python에서 집계된 비용 데이터를 기반으로
Excel의 Pie Chart를 자동 생성하여 비용 구조를 시각화하였습니다.
RPA는 Excel 차트 객체를 생성하고
데이터와 연결하는 작업을 자동으로 수행합니다.
자동화 과정은 다음과 같습니다.

1. 비용 항목별 데이터 범위를 자동으로 인식합니다.
2. Pie Chart 객체를 생성합니다.
3. 범례와 데이터 레이블을 자동 적용합니다.
4. 차트의 크기와 위치를 보고서 레이아웃에 맞게 정렬합니다.

이를 통해 기업의 비용 구조를 직관적으로 파악할 수 있는
시각화 리포트를 자동 생성하도록 구현하였습니다.

RPA



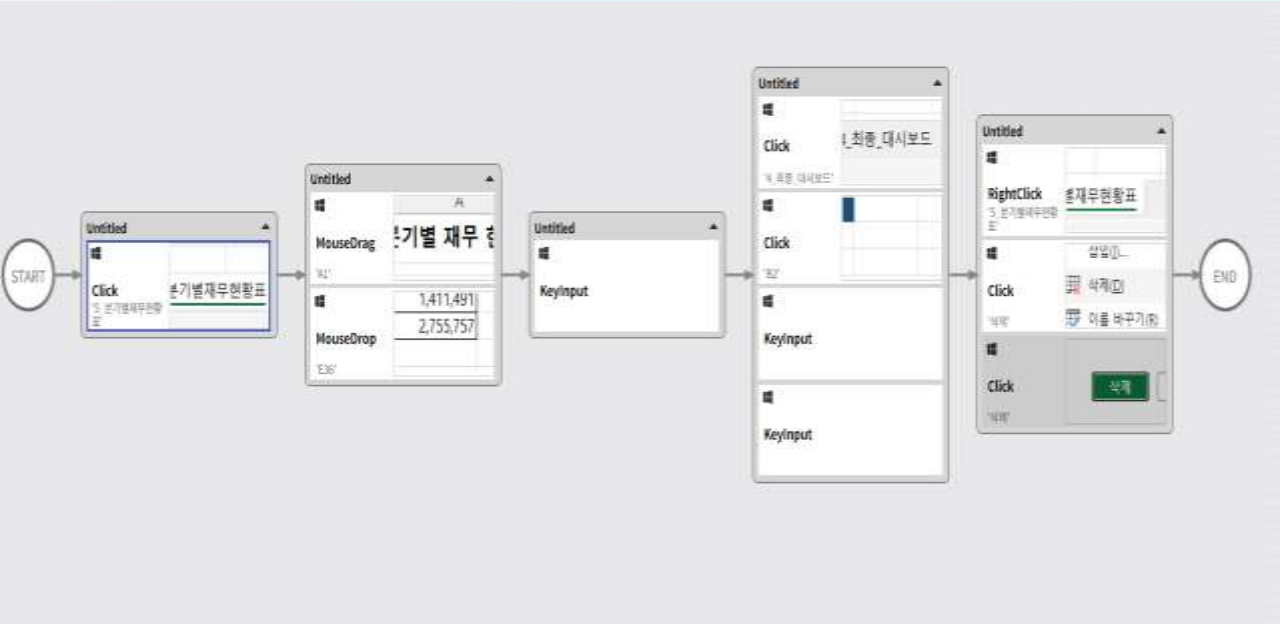
4.포괄손익계산서

Python에서 계산된 포괄손익 데이터를 기반으로
Excel 차트를 자동 생성하여 재무 흐름을 시각화하였습니다.
RPA는 Excel 애플리케이션을 제어하여
다음과 같은 자동화 기능을 수행합니다.

- 차트 객체 자동 생성
- 데이터 범위 자동 연결
- 차트 스타일 적용
- 보고서 레이아웃에 맞는 자동 배치

이 과정을 통해 재무 데이터를 시각적으로 전달할 수 있는
대시보드 요소를 자동 생성하도록 구현하였습니다.

RPA



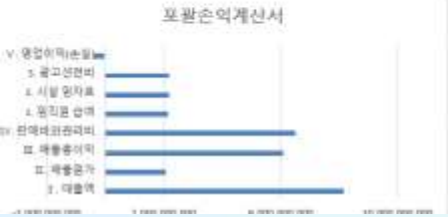
5.분기별 재무 현황표 옮기기

Python에서 생성된 재무 현황 데이터를
최종 보고서 Excel 템플릿으로 자동 이관하도록 구현하였습니다.
RPA는 Excel Workbook을 제어하여
다음 작업을 수행합니다.

1. Python 결과 파일을 로드합니다.
2. 필요한 시트와 데이터 범위를 선택합니다.
3. 보고서 템플릿 시트로 데이터를 복사합니다.
4. 기존 서식과 레이아웃을 유지한 상태로 데이터를 배치합니다.

이를 통해 데이터 분석 결과를
경영진 보고서 형식으로 자동 변환하는 기능을 구현하였습니다.

RPA



분기별 재무 현황표

손익계산서 (Income Statement)

구분	2024			
구분	Q1	Q2	Q3	Q4
매출액	2,126,378	1,968,861	2,056,892	1,984,800
매출원가	541,717	518,918	489,163	511,275
매출총이익	**1,584,661**	**1,449,943**	**1,567,729**	**1,473,525**
판매비와관리비	1,579,057	1,673,239	1,562,521	1,689,176
영업이익	**5,604**	**-223,296**	**5,208**	**-215,651**
영업외수익	224	-8,932	208	-8,626
영업외비용	84	-3,340	78	-3,235
법인세차감전순이익	**5,744**	**-228,879**	**5,338**	**-221,042**
법인세비용	1,264	-50,353	1,174	-48,629
당기순이익	**4,480**	**-178,526**	**4,164**	**-172,413**

재무상태표 (Balance Sheet)

구분	2024			
구분	Q1	Q2	Q3	Q4
자산 총계	**4,298,363**	**4,001,399**	**4,200,923**	**4,104,124**
유동자산	4,298,363	4,001,399	4,200,923	4,104,124
비유동자산	0	0	0	0
부채 총계	**0**	**0**	**0**	**0**
유동부채	0	0	0	0
비유동부채	0	0	0	0
자본 총계	**4,298,363**	**4,001,399**	**4,200,923**	**4,104,124**

현금흐름표 (Cash Flow Statement)

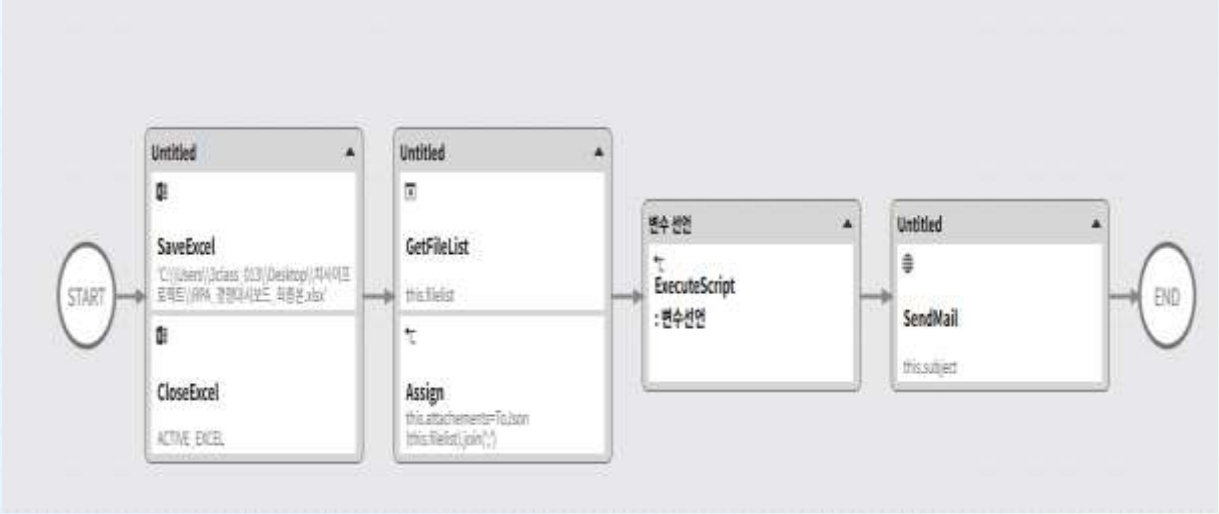
구분	2024			
구분	Q1	Q2	Q3	Q4
영업활동 현금흐름	5,604	-223,296	5,208	-215,651
투자활동 현금흐름	0	0	0	0
재무활동 현금흐름	0	0	0	0
현금의 순증감	**5,604**	**-223,296**	**5,208**	**-215,651**
기초 현금	0	1,433,642	1,226,568	1,411,491
기말 현금	1,433,642	2,660,210	2,638,059	2,755,757

PYTHON,RPA를 활용한 결과물

PYTHON과 RPA로 가공한 전체 결과물

월별손익,비용구조,포괄손익,분기별 재무 현황표

RPA



변수

추가	이름	초기값	값 형식	범위	설명
삭제	this.filelist	ex) null	Any	Normal	겟파일리스트 가져온 파일리스트
삭제	this.attache···	ex) null	Any	Normal	첨부파일
삭제	this.str_cc	ex) null	Any	Normal	메일 참조자
삭제	this.str_id	ex) null	Any	Normal	메일 아이디
삭제	this.password	ex) null	Any	Normal	메일 비밀번호
삭제	this.sender	ex) null	Any	Normal	메일 보내는사람
삭제	this.receivers	ex) null	Any	Normal	메일 받는 사람
삭제	this.subject	ex) null	Any	Normal	메일 제목
삭제	this.body	ex) null	Any	Normal	메일 내용

```
// 한줄 주석
/*
다중 주석
*/

//아이디값 입력
this.str_id = 'jongbin344'
//앱 비밀번호
this.password = 'klucsoanyzgpusec'
//발신자 (보내는사람 주소)
this.sender = 'jongbin344@gmail.com'
//수신자 (받는사람)
this.receivers = 'fprtm5390@gmail.com'
//참조자
this.str_cc = 'aig0207sw@gmail.com'
//메일제목
this.subject = '스몰프로젝트'
//메일내용
this.body = '최종 스몰프로젝트 '
```

엑셀 저장 및 메일링

최종 보고서가 생성된 이후

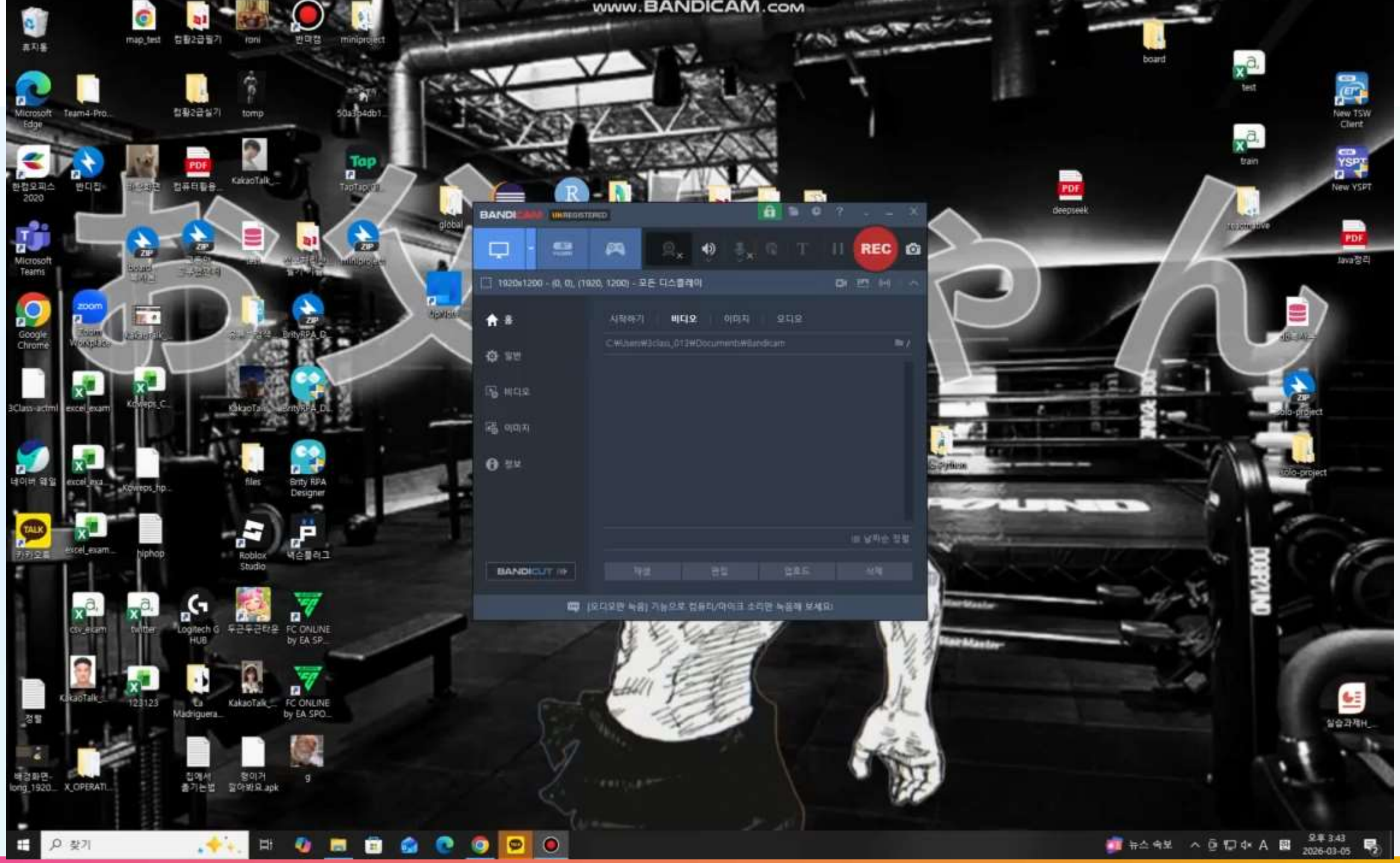
RPA는 메일 서버(SMTP)와 연동하여 결과 리포트를 자동으로 배포합니다.

자동화 프로세스는 다음과 같습니다.

1. 최종 Excel 보고서를 저장합니다.
2. 수신자와 참조자를 자동으로 설정합니다.
3. 메일 제목과 본문을 자동으로 작성합니다.
4. Excel 보고서를 첨부합니다.
5. SMTP 서버를 통해 메일을 자동 발송합니다.

이를 통해 재무 분석 → 보고서 생성 → 메일 배포까지

완전 자동화된 End-to-End 업무 프로세스를 구현하였습니다.


www.BANDICAM.com
BANDICAM
UNREGISTERED
REC
1920x1200 - (0, 0), (1920, 1200) - 모든 디스플레이
홈
일반
비디오
이미지
정보
시작하기
비디오
이미지
오디오
BANDICUT
(오디오만 녹음) 기능으로 컴퓨터/마이크 소리만 녹음해 보세요!
휴지통
map_test
컴활2급필기
roni
반마켓
miniproject
board
test
train
deepseek
New TSW Client
New YSPT
Java정리
Microsoft Edge
Team4-Pro...
컴활2급실기
tomp
한컴오피스 2020
반디캡
컴퓨터활용...
KakaoTalk_...
TapTap
global
Microsoft Teams
Google Chrome
zoom
3Class-actml
excel_exam
Kowaps_C...
excel_exa
Kowaps_hp...
files
Brity RPA Designer
카카오톡
excel_exam...
hiphop
Roblox Studio
넥슨플러그
csv_exam
twitter
Logitech G HUB
두근두근타운
FC ONLINE by EA SP...
KakaoTalk_...
123123
La Madriguera...
KakaoTalk_...
FC ONLINE by EA SPO...
정렬
배경화면-long_1920...
X_OPERATI...
집에서 즐기는법
장미가 알아봐요.apk
g
solo-project
실습과제H_...
찾기
뉴스 속보
오후 3:43
2026-03-05

프로젝트 성과 및 배운 점

기술적 발전

Before

- 단순한 엑셀 수작업 및 단편적인 파이썬 스크립트 작성에만 익숙했음
- 대용량 데이터 처리 및 이기종 시스템 간의 연동 경험 부족

After

- Pandas를 활용해 1만건 이상의 비정형 전표 데이터를 고속으로 가공하는 ETL 파이프 구축 능력 확보
- Python과 RPA의 역할을 명확히 분리하는 하이브리드 자동화 아키텍쳐 설계 및 상태 관리 능력 향상

팀워크에서 배운 점

배운 점

- Python과 RPA간 데이터를 주고 받을 때 무결점 데이터 양식과 오류플래그등 명확한 인터페이스 합의가 필수적임을 깨달음
- 오류 발생시 혼자 고민하기보다 데이터파트와 RPA파트가 함께 데이터 흐름을 추적하는 것이 효율적임을 체감

팀 성과

- 재무 자동화 핵심 프로세스 100% 구현: 원천 데이터 파싱 ➔ 재무제표 서식 렌더링 ➔ 동적 차트 생성 ➔ 경영진 메일 발송까지 이어지는 완벽한 Zero-Touch 파이프라인 완성.

아쉬운 점 개선 방향

아쉬운 점

- 초기 원천 데이터(CSV)에 예상치 못한 결측치나 오타가 있을 때, RPA가 완벽하게 우회할 수 있도록 더 정교한 예외 처리(Error Handling) 로직을 적용하지 못한 점.

개선 방향

- 현업의 다양한 예외상황(누락,양식변경등)에 대해 대비한 동적 방어코드 추가 및 에러로깅 시스템 고도화
- 실제 경영진 피드백을 가정한 대시보드 UI/UX 추가개선

자체평가(느낀점, 아쉬운점)

🏆 김강산

- 단순한 스크립트 코딩을 넘어, 실제 기업 재무 기준을 시스템 로직으로 완벽히 구현해 낸 것이 이번 프로젝트의 가장 큰 성과입니다.
  매출총이익에서 영업이익, 순이익으로 이어지는 계층적인 이익 산출 과정을 코드로 설계하며, 개발자로서 비즈니스 도메인 지식을 이해하고 시스템에 녹여내는 역량의 중요성을 체감했습니다.
  1만 건 이상의 로우 데이터를 처리하면서, 단순히 데이터를 나열하는 것에 그치지 않고 후행 작업인 RPA가 즉시 사용할 수 있도록 최적화된 '데이터 마트'를 구축했습니다. 결측치 제어, 시계열 데이터 파싱 등 꼼꼼한 전처리 과정을 거치며 데이터의 무결성이 곧 시스템 전체의 신뢰도로 이어진다는 것을 깊이 배웠습니다.
  또한, Python이 무거운 '데이터 연산'을 전담하고 RPA가 '시각화 및 메일 발송'을 담당하도록 아키텍처를 분리했습니다. 이 과정에서 다른 기술 스택을 담당하는 팀원과 명확한 데이터 인터페이스 규격을 합의하고, 오류 발생 시 즉각적으로 상태를 공유하는 플래그(Error Flag) 구조를 설계하며, 이기종 시스템 간의 매끄러운 연동과 소통 기반의 협업 시너지를 제대로 경험할 수 있었습니다.
  다만, 초기 원천 데이터에 예상치 못한 포맷 오류나 심각한 결측치가 발생했을 때, 이를 유연하게 우회하거나 관리자에게 정교하게 리포팅하는 '동적 에러 로깅시스템'을 더 깊게 구현하지 못한 점은 아쉬움으로 남습니다.
  향후 실무에서는 로컬 파일 기반의 처리를 넘어 실제 DB 연동 및 클라우드 환경을 고려한 더욱 견고한 데이터 파이프라인으로 확장해 보고 싶습니다.

🏆 윤종빈

- RPA와 파이썬을 활용한 이번 프로젝트는 단순한 과제 수행을 넘어 자동화 설계 역량을 한 단계 끌어올릴 수 있었던 의미 있는 경험이었습니다. 이전 프로젝트에서 발생했던 오류와 비효율적인 구조를 되돌아보며 문제의 원인을 분석했고, 개선이 필요한 부분을 사전에 정리한 뒤 이번 프로젝트에 반영했습니다. 특히 기획 단계에서 전체 업무 프로세스를 흐름도 기반으로 구조화하여 자동화 대상 구간을 명확히 정의했고, RPA와 Python의 역할을 구분해 설계함으로써 개발 과정의 혼선을 줄이고 효율성을 높였습니다.

  데이터 수집 및 전처리 과정에서는 다양한 예외 상황을 고려해 안정성을 강화했으며, 반복적으로 사용되는 로직은 함수 단위로 모듈화하여 가독성과 재사용성을 확보했습니다. 또한 Python에서 생성된 결과 데이터를 RPA 프로세스와 유기적으로 연동시키기 위해 인터페이스 구조를 표준화하고, 데이터 이관 과정에서 발생할 수 있는 오류를 최소화하도록 검증 로직을 보완했습니다.

  이번 프로젝트를 통해 단순히 기능을 구현하는 수준을 넘어, 확장성과 유지보수성을 고려한 구조적 설계의 중요성을 체감할 수 있었습니다. 더 나아가 자동화 도구를 실무 환경에 적용하기 위해서는 기술 구현뿐 아니라 프로세스 이해와 리스크 관리 역량이 함께 요구된다는 점을 배우게 되었습니다. 결과적으로 자동화 설계 능력과 문제 해결 역량 모두에서 한 단계 성장할 수 있었던 프로젝트였다고 생각합니다.

자체평가(느낀점, 아쉬운점)

🏆 김찬

이번 미니 프로젝트를 통해 단순 반복 업무를 자동화하는 것이 업무 효율에 얼마나 큰 영향을 주는지 체감할 수 있었습니다. 파이썬 엔진을 활용해 분기별 IS, BS, CF에서 핵심 지표를 추출하고 재무제표 생성하는 로직을 설계하면서, 단순히 데이터를 처리하는 수준을 넘어 의미 있는 정보로 가공하는 과정의 중요성을 이해하게 되었습니다. 또한 메일 서버와 연동하여 결과 엑셀 파일을 자동으로 발송하도록 구현하면서, 실제 기업 환경에서 자동화가 어떻게 실무에 적용되는지를 경험할 수 있었습니다. 특히 지정된 폴더에 새로운 전표 CSV 파일이 업로드되면 자동으로 분석 프로세스가 시작되는 이벤트 기반 구조를 설계한 부분은, 수동 개입을 최소화하는 완전 자동화 파이프라인의 가능성을 확인한 의미 있는 경험이었습니다. 이번 미니 프로젝트를 통해 RPA와 파이썬을 결합하면 재무 분석, 보고서 작성, 배포까지 하나의 흐름으로 연결할 수 있다는 점에서 실무 활용 가치가 크다고 느꼈습니다.

아쉬운 점
아쉬운 점이라면 우선 재무 요약 문장 생성 로직이 정형화된 규칙 기반 방식에 머물러 있어, 다양한 예외 상황이나 특이 케이스를 충분히 반영하지 못했다는 한계가 있었습니다. 예를 들어 특정 계정과목의 급격한 변동이나 비정상 수치에 대한 해석이 아직 자동화 단계에서 정교하지 못했습니다. 또한 메일 발송과 스케줄링 기능은 정상 동작했지만, 장애 발생 시 로그 관리나 재시도 로직이 체계적으로 설계되지 못한 점도 개선이 필요하다고 느꼈습니다. 이벤트 트리거 방식 역시 단순히 파일 업로드 여부만 감지하도록 구성되어, 데이터 형식 오류나 누락에 대한 사전 검증 로직이 부족했다는 점이 아쉬웠습니다. 앞으로 AI 기반 요약 문장 생성, 데이터 검증 로직 강화, 예외 처리 자동화까지 확장하고, 그래프 디자인, 표 디자인 등 보고서 디자인을 좀 더 깔끔하게 만든다면 더 완성도 높은 결산 자동화 시스템이 될 것이라고 생각합니다.

Python & RPA 기반 대용량 재무 데이터 분석

감사합니다

김강산,김찬, 윤종빈

작성일 2026.03.10